Universidad Nacional de Asunción - Facultad Politécnica

Prof: Ing. Gustavo Sosa Cataldo
Fecha: 19/10/2026

Actividad: Implementación de caso práctico usando Flutter

Reglas

- No se puede utilizar herramientas de IA: **ni web, ni agentes en el IDE** (Claude Code, Codex, Copilot, etc.).
- Tiempo de desarrollo: **60 minutos**.
- Utilizar el adjunto facilitado por el profesor **ventas-api.zip**: proyecto Node puro que corre directamente con `node server.js`.
- Maximo 2 alumnos por grupo

1. Tema

Desarrollar una aplicación Flutter de ventas que consuma la API provista: listar las ventas cargadas, ver el detalle de cada una y registrar nuevas ventas con un esquema maestro-detalle (una cabecera y varios ítems). Las pantallas se muestran a continuación de forma simplificada; se puede mejorar el diseño siempre que se respete la funcionalidad.

2. API provista (ventas-api)

Al ejecutar `node server.js` queda disponible en el puerto **4002**. Los datos viven en memoria.

Método	Ruta	Descripción
`GET`	`/ventas`	Devuelve el arreglo de todas las ventas con sus detalles.
`POST`	`/ventas`	Crea una venta. Responde 201 con la venta creada (con su id) o 400 si falta algún campo.

Venta:

```
{ "id": 1, "fecha": "10/08/2026", "cliente": "Juan Perez", "total": 59000,
  "detalles": [ { "cantidad": 2, "descripcion": "Zapato",
                  "precio_unitario": 22000, "total_detalle": 44000 } ] }
```

El id lo asigna el servidor. En el POST se envían fecha, cliente, total y detalles (cada detalle con cantidad, descripcion, precio_unitario y total_detalle); todos son obligatorios y detalles debe tener al menos un elemento.

Pruebas con curl:

```
curl http://localhost:4002/ventas

curl -X POST http://localhost:4002/ventas -H "Content-Type: application/json" \
  -d '{"fecha":"19/10/2026","cliente":"Maria Lopez","total":34000,"detalles":[
    {"cantidad":1,"descripcion":"Remera","precio_unitario":25000,"total_detalle":25000},
    {"cantidad":3,"descripcion":"Medias","precio_unitario":3000,"total_detalle":9000}]}'
```

URL base según dónde se ejecute la app:

- Chrome, Windows, macOS: `http://localhost:4002`
- Emulador Android: `http://10.0.2.2:4002` | Celular físico: IP de la PC (misma red WiFi).

3. Pantallas a implementar

3.1 Listado de ventas (pantalla principal)

```
+----------------------------------+
| Ventas                           |
|----------------------------------|
| 10/08/2026                       |
| Juan Perez            Gs. 59.000 |
|----------------------------------|
| 01/09/2026                       |
| Diego Gonzalez       Gs. 120.000 |
|                                  |
|                                  |
|                            ( + ) |
+----------------------------------+
```

3.2 Detalle de una venta

```
+----------------------------------+
| Venta #1                         |
|----------------------------------|
| Fecha:   10/08/2026              |
| Cliente: Juan Perez              |
| Total:   Gs. 59.000              |
|----------------------------------|
| Cant Descripcion  P.Unit  Total  |
| 2    Zapato      22.000  44.000  |
| 3    Media        5.000  15.000  |
|                                  |
+----------------------------------+
```

3.3 Nueva venta (maestro-detalle)

```
+------------------------------------------------+
| Nueva venta                                    |
|------------------------------------------------|
| Fecha:   [ 19/10/2026 ]                        |
| Cliente: [ Juan Perez                  ]       |
| Total:   Gs. 59.000     (solo lectura)         |
|------------------------------------------------|
| Cantidad: [ 2 ]                                |
| Descripcion: [ Zapato               ]          |
| Precio unitario: [ 22000 ]        [ Agregar ]  |
|------------------------------------------------|
| Detalles agregados:                            |
| Cant Descripcion       P.Unit     Total        |
| 2    Zapato           22.000    44.000   [x]   |
| 3    Media             5.000    15.000   [x]   |
|                                                |
|                               [ Guardar venta ] |
+------------------------------------------------+
```

4. Requerimientos

- **Listado:** al abrir la app se cargan las ventas con GET /ventas (mostrar un indicador de carga). Cada ítem muestra fecha, cliente y total. Tocar un ítem abre el detalle (pantalla nueva o diálogo). El botón flotante (+) abre "Nueva venta".
- **Nueva venta:** la cabecera lleva fecha y cliente. El bloque de detalle pide cantidad, descripción y precio unitario; el botón Agregar añade la línea a la lista de detalles agregados (con opción de quitarla).
- **Cálculos:** total_detalle = cantidad × precio_unitario; el total de la cabecera es la suma de los total_detalle y se actualiza solo (no editable).
- **Validaciones:** cliente no vacío, cantidad y precio numéricos mayores a 0, y al menos un detalle para poder guardar.
- **Guardar:** envía POST /ventas, vuelve al listado y este se refresca mostrando la nueva venta. Si la API falla, mostrar un mensaje (SnackBar).
- **Se sugiere separar:** modelos (Venta y Detalle con fromJson/toJson), un servicio con el paquete http y las pantallas.

5. Criterios de evaluación

Criterio	Puntos
Listado de ventas (GET) con indicador de carga	15
Detalle de una venta	15
Formulario de nueva venta: cabecera + agregar/quitar detalles	25
Cálculo automático de totales y validaciones	15
Guardado (POST) y refresco del listado al volver	20
Orden del código (modelo, servicio, pantallas separadas)	10
Total	**100**